

Parallel Epson V39 Scanner System

Overview

This project enables reliable parallel scanning using multiple Epson Perfection V39 flatbed scanners by isolating each scanner inside its own virtual machine. Epson scanners do not behave reliably when accessed concurrently from a single operating system, even when using threading, multiprocessing, containers, or LXD. The approach that works consistently is to give each scanner exclusive USB ownership through QEMU/KVM virtual machines.

The system was originally designed for 8 scanners but can be reduced to 4 if needed. The scripts support both, as long as VM-to-scanner mapping is consistent. While highly efficient and accurate for running flatbed scanners in parallel, it takes some time to set up and debug. To help, I've included some rough documentation below. The biggest issue with this system is mapping each VM and Scanner USB address with the host, [debug.sh](https://github.com/0x00sec/debug.sh) may aid in this issue.

High-Level Structure

The host Linux machine runs virt-manager (QEMU/KVM).

Each Epson V39 scanner is passed through to exactly one VM.

Each VM runs a small Python script (scan.py) that:

- identifies its scanner
- resets the USB device
- performs a scan
- writes a TIFF output

The host coordinates all VMs using shell scripts and a tmux-based interface.

Requirements

Host system:

- Linux (Ubuntu recommended)
- KVM enabled
- virt-manager installed
- Epson V39 scanners connected directly via USB
- USB 3.0 ports preferred

Guest VM (all VMs identical):

- Ubuntu
- sane, sane-utils
- scanimage
- Python 3
- usbutils
- usbreset
- SSH enabled

VM Design and USB Considerations

USB 3.0 should be used whenever possible. V39 scanners generate large data transfers during scanning, and USB 2.0 significantly increases contention and latency.

If the host machine has multiple USB controllers (multiple buses), distribute scanners across them evenly. For example:

- 4 scanners on USB bus A
- 4 scanners on USB bus B

Avoid placing all scanners on a single USB controller. Even with VM isolation, shared USB buses can become a bottleneck. When possible, plug scanners into physically different ports that map to different controllers.

If using multiple USB buses, consider staggering scan start times slightly or batching scans per bus to reduce peak contention.

Creating the Base VM

Create a single base VM in virt-manager.

Suggested configuration:

- Name: scanner-base
- OS: Ubuntu
- User: Up to you
- Password: Up to you
- No USB devices attached

Inside the VM, install dependencies:

```
sudo apt update
```

```
sudo apt install sane sane-utils usbutils python3
```

Place scan.py into the VM, for example:

```
/home/seeds scanner/scan.py
```

Do not attach a scanner to the base VM.

Cloning the Scanner VMs

Clone scanner-base to create one VM per scanner.

Example naming scheme:

- scanner-blue
- scanner-orange
- scanner-gray
- scanner-green
- scanner-white
- scanner-black
- scanner-yellow
- scanner-crimson

The exact names are not required, but consistent naming helps when debugging. Each VM will use its hostname internally to identify itself.

USB Scanner Assignment

For each scanner VM:

1. Open the VM in virt-manager
2. Open hardware details
3. Add a USB Host Device
4. Select exactly one Epson V39 scanner
5. Ensure no other VM uses the same scanner

Each scanner must be attached to one and only one VM.

If scanners fail to appear or behave inconsistently, fully power down your host machine and power everything back up after reattaching devices.

Scanner ID Verification

Inside a VM, verify scanner visibility:

```
scanimage -L
```

To manually test a specific scanner device:

```
scanimage -d 'epsonscan2:Perfection V39/GT-S650:001:013:esci2:usb:ES010D:317' \  
--format=tiff --mode=Color --resolution 75 --source Flatbed > /dev/null
```

Change the device ID as needed.

How scan.py Works

Each VM runs scan.py independently.

Behavior:

- Terminates any lingering scanimage processes
- Detects the attached Epson V39 via scanimage -L
- Performs a USB reset on the device
- Runs a 1200 DPI color flatbed scan
- Writes output to /output/*.tiff

The VM hostname is used internally as the scanner identifier.

Running the System (Normal Workflow)

On the host machine:

1. Open a terminal
2. Change into the project directory
3. Run:

`./launch.sh`

`launch.sh` performs the following:

- starts all scanner VMs
- opens a tmux session
- splits into two panes (Batch 1 and Batch 2)
- runs the batch controller scripts
- shuts down VMs automatically when finished

This workflow is designed to be triggered via QR codes, but can also be run manually.

Debugging and Maintenance

To inspect USB mappings and scanner status:

```
./debug.sh
```

To clean VM disk usage and remove old scans:

```
./cleaner.sh
```

To inspect VM disk sizes:

```
sudo du -h /var/lib/libvirt/images/*.qcow2
```

Reducing the System to Fewer Scanners

The system works with fewer than 8 scanners.

For 4 scanners:

- use 4 VMs
- populate only Batch 1
- remaining scripts can be ignored

Ensure USB distribution is still balanced across controllers.

Final Notes

This system prioritizes reliability over simplicity. VM-based isolation combined with proper USB controller distribution is the only approach that consistently allows parallel Epson V39 scanning without corruption, lockups, or device contention.

If something breaks unexpectedly, the fix is usually:

- full power cycle
- rewire scanners across USB buses with `debugg.sh`

Once configured correctly, the system is stable and repeatable.